

-- AI(Claude)による評価 --

[Prompt] この研修コーステキストの、将来性、品質、市場価値を分類ごとにまとめてください。

研修コーステキスト 評価まとめ

対象コース：AWS インフラ / Azure インフラ / Java CI/CD / C# CI/CD / TypeScript CI/CD

1. AWS クラウドインフラ構築

評価軸	評価	内容
将来性	★★★★★	グローバルシェア約32%で首位。EC2・VPC・RDSなど基礎サービスの知識は長期間有効。AI/MLサービス拡充で需要増加中。
品質	★★★★★	VPC・サブネット・セキュリティグループ・IGWの構成を体系的に学習。本番環境と同等のアーキテクチャで実践的。
市場価値	★★★★★	AWS認定資格は国内外で最上位の評価。クラウドエンジニア・インフラエンジニアのキャリアに直結。

主要技術：VPC / EC2 / Security Group / PostgreSQL RDS / Ubuntu AMI

2. Azure クラウドインフラ構築

評価軸	評価	内容
将来性	★★★★★	Microsoft 365・Entra ID連携でエンタープライズ需要が急増。OpenAI Serviceなど生成AI基盤としても注目。
品質	★★★★☆	VNet・NSG・VM・Azure DB for PostgreSQLの構成を学習。AWSと比較しながら学べる点が付加価値。
市場価値	★★★★★	国内大企業・官公庁のMicrosoft採用率が高く、Azureスキルは就職・転職市場で強い競争優位。

主要技術：VNet / NSG / Virtual Machine / Azure DB for PostgreSQL / NIC / Public IP

3. TypeScript CI/CD パイプライン

評価軸	評価	内容
将来性	★★★★★	Webフロントエンド・BFF・サーバーサイドすべてで標準化が進行中。Node.jsランタイムの採用増加で今後も成長。

評価軸	評価	内容
品質	★★★★★	GitHub Actions + Node.js環境整備 + ビルド + AWSデプロイの一連フローを習得。現場即戦力の内容。
市場価値	★★★★★	フルスタックエンジニア需要の中心。Web系スタートアップ～大手ITまで幅広く求められるスキルセット。

主要技術： TypeScript / Node.js / GitHub Actions / AWS デプロイ

4. Java CI/CD パイプライン（JDK 21）

評価軸	評価	内容
将来性	★★★★☆	JDK 21はLTSリリースで安定。金融・基幹系システムでの長期需要は堅固だが、新興分野での採用はやや頭打ち。
品質	★★★★★	JDK 21インストール～ビルド～パッケージ転送の完全なパイプライン構成。Spring Boot等への応用が可能。
市場価値	★★★★☆	大手Sler・金融・製造業での求人は依然多い。エンタープライズ系キャリアを目指す場合は高い優位性。

主要技術： Java / JDK 21 LTS / GitHub Actions / AWS デプロイ

5. C# / ASP.NET Core CI/CD パイプライン

評価軸	評価	内容
将来性	★★★★☆	.NET 8 LTSで高いパフォーマンスを実現。AzureとのネイティブなCI/CD連携で、Microsoft技術スタック採用企業に特に有効。
品質	★★★★★	.NET SDK + ビルド + アーティファクト転送の完全パイプラインを学習。型安全性と静的解析による高品質開発が身につく。
市場価値	★★★★☆	Windows環境・Azureを採用する国内Sler・業務系SaaSで求人数多。Javaと並ぶエンタープライズ言語として安定。

主要技術： C# / ASP.NET Core / .NET SDK / GitHub Actions / AWS・Azure デプロイ

総合比較

コース	将来性	品質	市場価値	対象キャリア
AWS インフラ	◎	◎	◎	クラウドエンジニア
Azure インフラ	◎	○	◎	エンタープライズ系
TypeScript	◎	◎	◎	Webフルスタック

コース	将来性	品質	市場価値	対象キャリア
Java (JDK 21)	○	◎	○	Sler・金融・製造
C# / ASP.NET	○	◎	○	業務系SaaS・Azure

◎ = 最高評価 ○ = 高評価

総評

技術的正確性と実践性が高く、全コース研修資料として十分な水準。

- **AWSとTypeScript** は将来性・品質・市場価値の3軸すべてで最高評価。現代のクラウド開発・Web開発の中心として、幅広いキャリアに直結する。
- **Azure** も将来性・市場価値は最高評価。特に国内大企業・官公庁のMicrosoft環境では強みを発揮する。
- **JavaとC#** は品質が非常に高く実践的だが、新興分野での採用ペースと比較するとやや保守的な評価。ただしSler・金融・製造などエンタープライズ領域では今でも非常に強い需要がある。

-- HP紹介内容 --

研修名

GitHub Actionsハンズオン -- CI/CDの基本概念からパイプライン構築まで --

[P] この研修コースのデキアッチコピー

キャッチコピー

- コードを書けるから、システムも届けられるへ

研修概要

■ 本研修について

「コードは書けるけれど、インフラやデプロイは自信がない」——そんな課題を解消するために設計されたのが本研修です。クラウド環境の設計・構築からCI/CDパイプラインの実装まで、現場と同じ構成を使って手を動かしながら学びます。AWSとAzureの2つのクラウド、Java・C#・TypeScriptの3言語から自分の目標に合ったパターンを選択できるため、実務に即したスキルを効率よく習得できます。本研修を通じて、開発から運用環境へのデリバリーまでを一気通貫で担えるエンジニアを目指します。

■ 本研修の3つの目的

本研修では、以下の3つを習得することを目的としています。

- クラウドインフラ（AWS / Azure）を用いたネットワーク・サーバー・データベース環境の設計と構築
- GitHub Actionsを活用したCI/CDパイプラインの実装と、運用環境への自動デプロイ
- 開発から運用環境へのデリバリーまでを一気通貫で担える、実践的なエンジニアリング力の習得

■ 到達目標(受講後に身につくスキル)

- クラウド環境（AWS / Azure）でのネットワーク・サーバー・データベースの設計と構築
- セキュリティグループ・NSGを用いた、安全なインフラ構成の実装
- GitHub Actionsを活用したCI/CDパイプラインの設計・構築
- 選択した言語（Java / C# / TypeScript）によるビルドから運用環境への自動デプロイ
- 開発から運用環境へのデリバリーを一気通貫で担えるエンジニアリング力

こんな企業にお勧め

- AWS・Azureの導入・拡大フェーズにあり、インフラを自社で設計・運用できる人材を育成したい
- クライアントのクラウド移行案件に対応できるエンジニアを増やしたい
- 開発速度を上げるためにCI/CDを整備し、デリバリーの自動化を推進したい
- 既存システムのクラウド移行と合わせて、内製開発体制を強化したい

前提知識・スキル-> 受講にあたっての前提知識

プログラミングの基礎知識

- 受講パターンに応じて、Java（Spring Boot）・C#（ASP.NET Core）・TypeScript（Node.js）のいずれかを用いた、基本的なWebアプリケーションの作成・実行経験があること。

基本的なPC・OS操作

- Windows TerminalやPowerShellを用いた基本的なコマンド操作、ファイル編集ができること。
- Linux（Ubuntu）環境での初歩的なコマンド入力に抵抗がないこと。

Gitの基礎知識

- GitHubを使ったりリポジトリの作成・クローン・プッシュなど、基本的な操作ができること。

必要な受講環境

利用するクラウド環境

- **AWS** AWSマネジメントコンソール／東京リージョン（ap-northeast-1）／受講者ごとに用意されたAWSアカウント
- **Azure** Azureポータル／東日本リージョン（Japan East）／受講者ごとに用意されたAzureサブスクリプション

※ 受講パターンに応じて、いずれか一方を使用します。

アプリケーション開発環境

区分	環境 / ツール	役割・用途
ホストOS	Windows 11	研修用PCの基本環境
IDE	Visual Studio Code	コマンド操作およびアプリケーション開発
ターミナル	Windows Terminal / PowerShell	ファイル編集、コマンド操作、結果確認

区分	環境 / ツール	役割・用途
OS	Ubuntu 24.04	仮想サーバー（EC2 / VM）
RDBMS	PostgreSQL 17	クラウドデータベース（RDS / Azure DB）
SQLツール	pgAdmin 4	データベースリソース作成・管理
Java	JDK 21 / Spring Boot 4	Java Webアプリケーション構築
C#	.NET SDK 8 / ASP.NET Core 8	C# Webアプリケーション構築
TypeScript	Node.js 22 / TypeScript 5	TypeScript Webアプリケーション構築

※ Java / C# / TypeScript は受講パターンに応じていずれか一つを使用します。

日数

3日

カリキュラム

章・学習テーマ	学習内容・習得スキル
第1章：CI/CDとGitHub Actionsの概要	CI/CDの基本概念（継続的インテグレーション・継続的デリバリー）とそのメリットを学びます。GitHub Actionsの特徴と、Workflow・Event・Runner・Job・Step・Actionの6つの基本用語と階層構造を理解します。
第2章：ワークフローの基本構造	ワークフローファイルの配置場所（.github/workflows/）とYAML記法の基礎を学びます。5つの必須キーワード（name・on・jobs・runs-on・steps）の記述方法、push/手動/スケジュールによるトリガーの設定、実行履歴の確認と再実行の操作を実践します。
第3章：ワークフローの制御と環境設定	needsキーワードを使ったジョブの並列・直列実行の制御、actions/checkoutによるソースコード取得、多彩なトリガー（pull_request・schedule・workflow_dispatch等）とブランチ・パス指定によるフィルタリングを学びます。GitHubホスト型・セルフホスト型Runnerの違い、環境変数（env）とシークレット（secrets）による安全な設定管理を実践します。
第4章：CI（継続的インテグレーション）の概要	CIの目的（コード変更のたびにビルドとテストを自動実行し、バグを早期発見する仕組み）とメリットを理解します。ビルド・テスト・静的解析（Lint）の3本柱と、プルリクエストとCIを連携させてマージ前の品質を自動担保する仕組みを学びます。
第5章：C#アプリケーションのCI実践	ASP.NET Coreアプリを対象に、actions/setup-dotnetによる.NET SDK環境構築とdotnet buildによるビルドを自動化します。dotnet testによる単体テスト・結合テストの自動化、dotnet formatによる静的解析（フォーマット検証）、TestcontainersとDockerを使ったデータベース連動テストを段階的に構築します。
第6章：JavaアプリケーションのCI実践	Spring Boot×Gradleプロジェクトを対象に、actions/setup-javaとGradle Wrapper（./gradlew）を用いた再現性の高いビルド環境を構築します。JUnit 5による単体テスト、MockMvcを活用したWeb層の結合テスト、Spotlessによるコードフォー

章・学習テーマ	学習内容・習得スキル
	マット検証、TestcontainersとPostgreSQLを使ったDBアクセス層のテストを自動化します。
第7章：Next.js (TypeScript) アプリケーションのCI実践	Next.jsプロジェクトを対象に、actions/setup-nodeとキャッシュ (cache: 'npm') によるパッケージインストールの高速化を実装します。ESLint・Prettier・TypeScript型チェック (tsc --noEmit) による静的解析、Vitest+React Testing Libraryによるコンポーネントテスト、PlaywrightによるE2Eテストの自動化、MSW (Mock Service Worker) を使ったバックエンドAPI連携テストを構築します。
第8章：CDの概要とクラウドインフラ構築	CD (継続的デリバリー) の概念とBlue/Greenデプロイメントなどのデプロイ戦略を学びます。AWSとAzureの用語対照 (VPC/VNet・EC2/VM・RDS/Azure Database等)、セキュアなネットワーク設計 (パブリック/プライベートサブネット分離・最小権限のファイアウォール) を理解します。受講パターンに応じてAWS/Azureのインフラ環境を構築し、SSH・DB接続の疎通確認とGitHub Secretsへの認証情報登録を実践します。
第9章：C#アプリケーションのCDとワークフロー構築	dotnet publishによるLinux向け配布パッケージ (FDD/SCD) の生成、appleboy/ssh-actionを使った運用環境への接続と疎通確認、ASP.NET Coreランタイムの自動プロビジョニングを実装します。appleboy/scp-actionによるアーティファクト転送、SQLスクリプトによるデータベース初期化、Nginxリバースプロキシ設定、systemdによるアプリケーションのサービス化 (常駐化) とSecretを使ったDB接続情報の環境変数注入を構築します。
第10章：JavaアプリケーションのCDとワークフロー構築	GradleによるExecutable JAR (Fat JAR) の生成とパッケージング、appleboy/ssh-actionを使った運用環境への接続と疎通確認、OpenJDK 21の自動プロビジョニング (冪等性の確保) を実装します。appleboy/scp-actionによるJARファイル転送、SQLスクリプトによるデータベース初期化、Nginxリバースプロキシ設定、systemdによるアプリケーションのサービス化とSpring Boot環境変数オーバーライドによるセキュアな接続管理を構築します。
第11章：TypeScriptアプリケーションのCDとワークフロー構築	Next.jsのoutput: 'standalone'設定による最小フットプリントの配布パッケージ生成、appleboy/ssh-actionによる疎通確認、Node.js (LTS) とPM2の自動プロビジョニングを実装します。Standaloneディレクトリと静的アセット (public・.next/static) のSCP転送、NEXTAUTH_SECRET・NEXTAUTH_URL・API_BASE_URLの.env動的生成、PM2によるZero Downtime Reload (無停止再起動) とNginxリバースプロキシを構築します。
第12章：クラウドリソースのクリーニング	演習で作成したクラウドリソースを依存関係の順序に従って安全に削除します。AWSはEC2・RDS・VPC等の削除手順を、AzureはVM・Azure Database・VNet等の削除手順を実践し、不要なコストが発生しない運用管理の基本を学びます。

※ 第5・9章 (C#)、第6・10章 (Java)、第7・11章 (TypeScript) は受講パターンに応じていずれか一つを受講します。第8章のインフラ構築はAWS/Azureのいずれかを選択します。